

AWS Multi-Tier Student Management System Using EC2, RDS and VPC

A Dissertation submitted to

**BIGLEARN TRAINING INSTITUTE
TIRUCHIRAPPALLI**



In Partial Fulfilment of the Requirement for
The Award of the Degree of
CLOUD COMPUTING

Guided By

NANCY JOSHLIN.A

Submitted By

MOHAMMED RIYASDEEN M

ABSTRACT

- This project demonstrates the deployment of a web-based gaming application using Amazon Web Services (AWS).
- The application is hosted on an Amazon EC2 instance, while user and game-related data are stored in Amazon RDS. The system provides a scalable, secure, and highly available environment for online gaming applications. AWS services help reduce infrastructure management efforts and improve performance.
- The project highlights cloud computing concepts such as virtualization, database management, and web hosting.

INTRODUCTION

Cloud computing has transformed the way applications are developed and deployed. Gaming applications require high availability, scalability, and reliable data storage. AWS offers services that enable developers to deploy applications efficiently.

This project focuses on hosting a gaming application on AWS using:

- Amazon EC2 for application hosting
- Amazon RDS for database management
- Security Groups for access control
- VPC for network isolation

The project demonstrates practical implementation of cloud technologies in gaming platforms.

OBJECTIVES

Primary Objectives

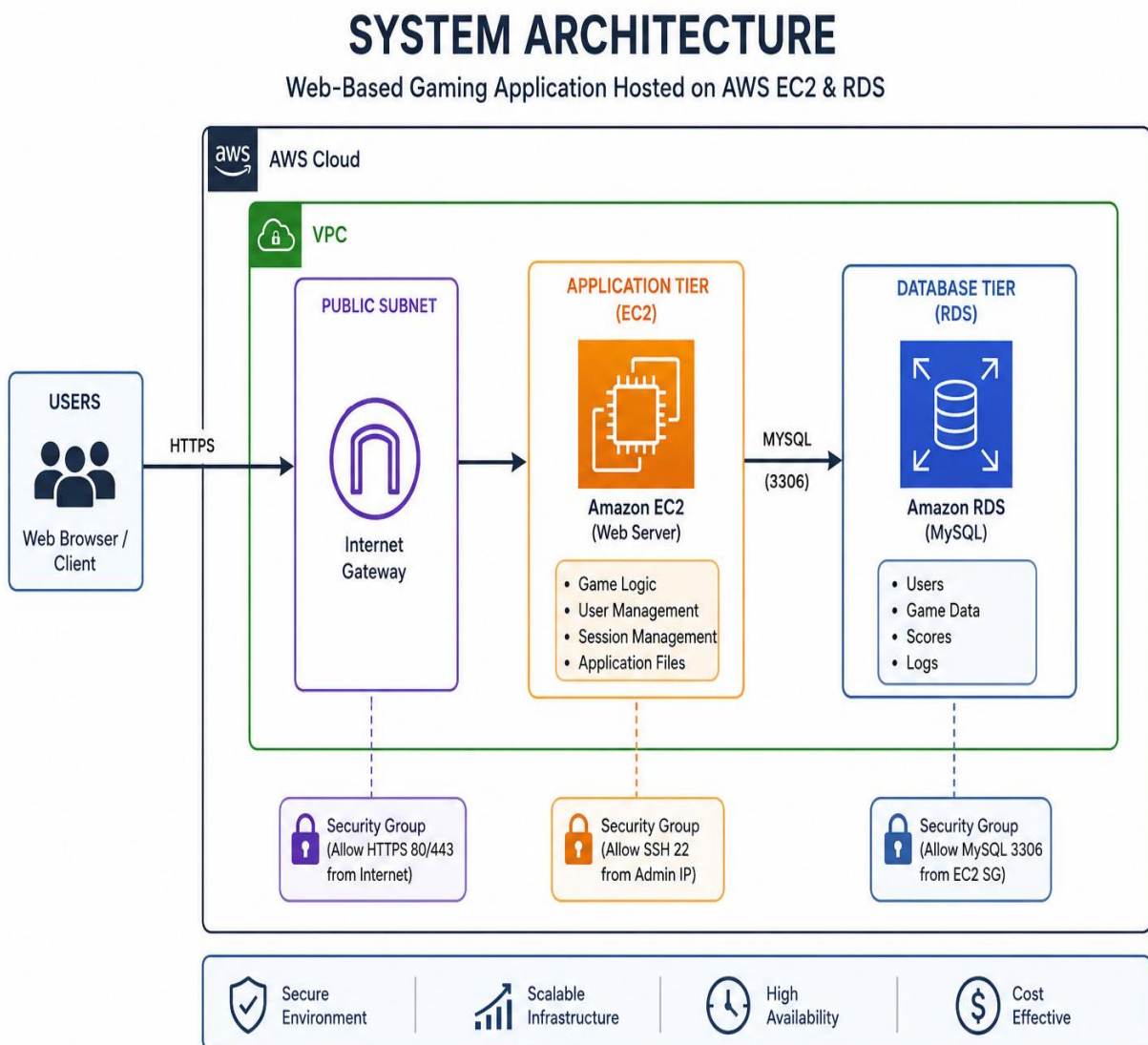
- Deploy a web-based gaming application on AWS.
- Store user and game data securely in Amazon RDS.
- Ensure high availability and scalability.

- Improve application performance through cloud infrastructure.

Secondary Objectives

- Learn AWS cloud services.
- Understand server deployment.
- Configure networking and security settings.

System Architecture



WORKFLOW

1. User accesses the gaming website.
2. Request reaches the EC2 server.
3. EC2 processes game logic.
4. Data is stored and retrieved from RDS.
5. Results are displayed to users.

TECHNOLOGIES USED

Hardware Requirements

- Laptop/Desktop
- Internet Connection

Software Requirements

- AWS Management Console
- Amazon EC2
- Amazon RDS
- Linux Operating System
- Web Browser

Cloud Services

- Amazon EC2
- Amazon RDS
- AWS VPC
- Security Groups

IMPLEMENTATION

Step 1: Create EC2 Instance

- Launch Linux EC2 instance.
- Configure Security Groups.
- Connect through SSH.

Step 2: Create RDS Database

- Launch MySQL RDS instance.
- Configure database credentials.
- Enable connectivity with EC2.

Step 3: Deploy Gaming Application

- Install required packages.
- Upload application files.
- Configure database connection.

Step 4: Testing

- Verify website accessibility.
- Test user registration and game functionality.

RESULT AND OUTPUT

The gaming application was successfully deployed on AWS infrastructure.

Achievements

- Successful hosting on EC2.
- Database connectivity with RDS.
- Secure network configuration.
- Accessible through public IP address.

Benefits

- Scalability
- Reliability
- Cost-effectiveness
- Better performance

✔ Success

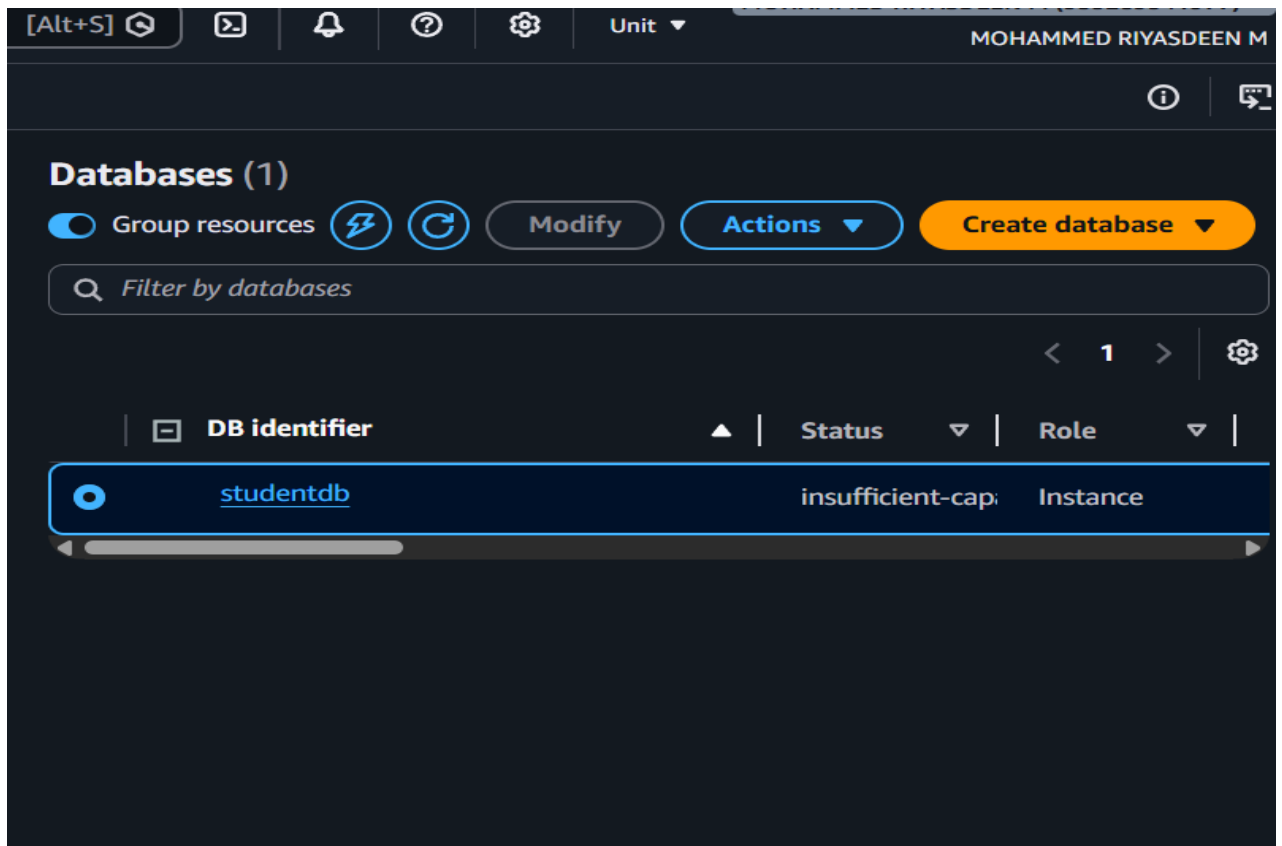
- ✔ Create VPC: `vpc-086f79fa25d21e364` [↗](#)
- ✔ Enable DNS hostnames
- ✔ Enable DNS resolution
- ✔ Verifying VPC creation: `vpc-086f79fa25d21e364` [↗](#)
- ✔ Create subnet: `subnet-0a8a372b3576827c8` [↗](#)
- ✔ Create subnet: `subnet-0f5a86c3e899b8e06` [↗](#)
- ✔ Create internet gateway: `igw-0afb27a36a4d7ea2f` [↗](#)
- ✔ Attach internet gateway to the VPC
- ✔ Create route table: `rtb-04805c1623c15bd6a` [↗](#)
- ✔ Create route
- ✔ Associate route table
- ✔ Create route table: `rtb-07206a9ea187ef923` [↗](#)
- ✔ Associate route table
- ✔ Verifying route table creation

Discover additional resources you can launch within your VPC to build your network architecture. From security components to connectivity services, explore the building blocks available for your infrastructure.

```

#_
~\#### Amazon Linux 2023
~~\#####\
~~\####|
~~\#/ https://aws.amazon.com/linux/amazon-linux-2023
~~V~' '->
~~~
~~~
~~~
~/m/' -
Last login: Mon Jun 22 11:24:23 2026 from 18.206.107.29
[ec2-user@ip-172-31-15-177 ~]$ curl http://localhost:5000
curl: (7) Failed to connect to localhost port 5000 after 0 ms: Could not connect to server
[ec2-user@ip-172-31-15-177 ~]$ ls
app.py  ec2file.txt  newfile.txtc
[ec2-user@ip-172-31-15-177 ~]$ python3 app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production W
I server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.31.15.177:5000
Press CTRL+C to quit

```



```
-> ('Harish','harish@example.com'),
-> ('Bala','bala@example.com'),
-> ('Vijay','vijay@example.com'),
-> ('Sathish','sathish@example.com'),
-> ('Akash','akash@example.com'),
-> ('Ramesh','ramesh@example.com'),
-> ('Saravanan','saravanan@example.com'),
-> ('Gokul','gokul@example.com'),
-> ('Mohan','mohan@example.com'),
-> ('Ashwin','ashwin@example.com'),
-> ('Keerthan','keerthan@example.com'),
-> ('Abdul','abdul@example.com'),
-> ('Faizal','faizal@example.com');
Query OK, 25 rows affected (0.02 sec)
Records: 25  Duplicates: 0  Warnings: 0

mysql> SELECT COUNT(*) FROM students;
+-----+
| COUNT(*) |
+-----+
|      26 |
+-----+
1 row in set (0.00 sec)

mysql> exit
Bye
```



Student Records

ID	Name	Email
1	Riyasdeen	riyas@example.com
2	Arun	arun@example.com
3	Karthik	karthik@example.com
4	Vignesh	vignesh@example.com
5	Praveen	praveen@example.com
6	Sanjay	sanjay@example.com
7	Rahul	rahul@example.com
8	Ajith	ajith@example.com
9	Surya	surya@example.com
10	Dinesh	dinesh@example.com
11	Manoj	manoj@example.com
12	Kumar	kumar@example.com
13	Naveen	naveen@example.com
14	Harish	harish@example.com
15	Bala	bala@example.com
16	Vijay	vijay@example.com
17	Sathish	sathish@example.com
18	Akash	akash@example.com
19	Ramesh	ramesh@example.com
20	Saravanan	saravanan@example.com
21	Gokul	gokul@example.com
22	Mohan	mohan@example.com
23	Ashwin	ashwin@example.com
24	Keerthan	keerthan@example.com
25	Abdul	abdul@example.com
26	Faizal	faizal@example.com

RESULTS

The application successfully displays student records stored in Amazon RDS through a Flask web interface hosted on AWS EC2.

Output Features

- Displays all student records.
- Retrieves data dynamically from MySQL.
- Accessible through public IP.
- Demonstrates cloud-hosted database connectivity.

ADVANTAGES

- Cloud-based deployment.
- Easy database management.
- Scalable infrastructure.
- Secure communication.
- Cost-effective for learning environments.

FUTURE ENHANCEMENTS

- Student login system.
- Admin dashboard.
- Add, Edit, Delete functionality.
- Search and filtering.
- AWS S3 integration for file storage.
- Load Balancer and Auto Scaling support.

CONCLUSION

The Web-Based Student Record Management System successfully demonstrates cloud application deployment using AWS services. By integrating Amazon EC2, Flask, and Amazon RDS, the project provides a reliable and scalable platform for managing student records. This project enhances practical knowledge of cloud computing, web development, and database management.

